

Chatbot Amendis

Documentation technique de mise en place du projet

Environnement, architecture, explication de Rasa et du RAG,
et description détaillée de chaque fichier du projet

Réalisé par : **Diyae ZIYANI**

Encadré par : Mr. Otman TRIATE

Stage d'observation — Juillet 2026

1. Vue d'ensemble du projet

L'objectif est de développer un **Proof of Concept (PoC)** de chatbot d'assistance client pour Amendis (distribution d'eau et d'électricité à Tanger et Tétouan). Le bot combine deux « cerveaux » complémentaires :

- **Un cerveau à règles strictes (Rasa)** : il reconnaît les demandes métier prévues (consulter une facture, payer, signaler une fuite...) et exécute des procédures déterministes, notamment l'interrogation d'une base de données locale SQLite qui simule le système d'information client d'Amenadis.
- **Un cerveau documentaire (RAG)** : quand la question sort des scénarios prévus (« comment résilier mon contrat ? », « comment est calculée ma consommation ? »), un système de *Retrieval-Augmented Generation* cherche la réponse dans un corpus constitué par scraping du site public **www.amendis.ma**, et la formule en langage naturel via un LLM local.

Adaptation par rapport au cahier des charges initial : la source documentaire n'est pas un ensemble de fichiers PDF mais le contenu du site web public d'Amenadis. Le second site, **www.amendisclient.ma** (espace client), est une application JavaScript accessible uniquement après connexion : il ne peut pas servir de source documentaire. Il joue donc le rôle du **système que notre base SQLite simule**, et le bot y **redirige** l'utilisateur pour les actions réelles (paiement en ligne).

Règle de sécurité fondamentale

Les données clients (numéros de contrat, montants de factures) sont traitées **uniquement en local** par le code Python et SQLite. Elles ne transitent **jamais** par le LLM. Seul le texte des questions générales (sans données personnelles) est envoyé au pipeline RAG — qui tourne lui aussi entièrement en local.

2. Ce qui a été téléchargé et installé

2.1. Python 3.10.11

Aucun Python n'était installé sur la machine. Nous avons installé **Python 3.10.11** (via le gestionnaire de paquets Windows `winget`). Ce choix de version n'est pas anodin : **Rasa 3.6 ne fonctionne qu'avec Python 3.8 à 3.10**. Une version plus récente (3.12, 3.13) aurait empêché l'installation de Rasa.

2.2. Deux environnements virtuels — pourquoi ?

Un **environnement virtuel** (venv) est une copie isolée de Python avec ses propres bibliothèques : cela évite que les dépendances d'un projet entrent en conflit avec celles d'un autre. Ici, nous avons dû créer **deux** venvs dans le même projet, pour une raison technique précise :

Conflit pydantic v1 / v2. Rasa 3.6 exige la bibliothèque `pydantic` en version 1, alors que LangChain (le framework du RAG) exige `pydantic` version 2. Les deux ne peuvent pas coexister dans le même environnement. La solution : deux venvs séparés, et le RAG exposé comme un **micro-service HTTP local** que Rasa appelle. Cette contrainte technique rejoint d'ailleurs l'exigence du cahier des charges : « *séparation de la logique métier et de l'IA générative* ».

2.3. venv_rasa — le moteur de dialogue

Bibliothèque	Rôle
<code>rasa 3.6.21</code>	Framework open source complet de chatbot : compréhension du langage naturel (NLU), gestion du dialogue, serveur API. C'est la dernière version open source stable.
<code>rasa-sdk 3.6.2</code>	Kit de développement pour écrire les <i>Custom Actions</i> (notre code Python métier, exécuté par un serveur d'actions séparé sur le port 5055).

2.4. venv_rag — le pipeline documentaire et l'interface

Bibliothèque	Rôle
<code>requests</code>	Effectuer les requêtes HTTP (téléchargement des pages du site Amendis).
<code>beautifulsoup4</code> + <code>lxml</code>	Analyser le HTML des pages téléchargées et en extraire le texte utile (en supprimant menus, pieds de page, scripts...).
<code>langchain</code> (+ <code>community</code> , <code>text-splitters</code>)	Framework qui orchestre le pipeline RAG : découpage des textes, connexion à la base vectorielle, construction du prompt, appel du LLM.
<code>sentence-transformers</code>	Modèles d' <i>embeddings</i> : transforment un texte en vecteur numérique qui capture son sens. Nous utilisons un modèle multilingue (compatible français) exécuté en local, sans envoyer de données dans le cloud.
<code>chromadb</code>	Base de données vectorielle : stocke les fragments de texte et leurs vecteurs, et retrouve en quelques millisecondes les fragments les plus proches d'une question.
<code>langchain-ollama</code>	Connecteur vers Ollama , le logiciel qui fait tourner un LLM en local.
<code>fastapi</code> + <code>uvicorn</code>	Créer et servir la petite API HTTP du service RAG (port 8000), appelée par Rasa.
<code>streamlit</code>	Framework Python pour créer l'interface web de chat sans écrire de HTML/JavaScript.

2.5. Reste à installer : Ollama

Ollama est un logiciel gratuit qui exécute des LLM open source directement sur le PC (sans clé API, sans envoyer de données à l'extérieur). Il faudra le télécharger depuis <https://ollama.com>, puis récupérer un modèle léger avec la commande `ollama pull llama3.2` (~2 Go). C'est ce modèle qui rédigera les réponses documentaires du RAG.

3. Comprendre Rasa

Rasa Open Source est un framework Python de référence pour construire des assistants conversationnels. Contrairement à un LLM type ChatGPT qui « improvise » ses réponses, Rasa est **déterministe** : il classe le message de l'utilisateur dans des catégories apprises, puis suit des règles de dialogue écrites par le développeur. C'est exactement ce qu'il faut pour les procédures sensibles (consultation de données clients), où l'on ne peut pas se permettre d'improvisation.

3.1. Les concepts clés

Concept	Définition	Exemple dans notre projet
Intention (intent)	La catégorie de ce que veut l'utilisateur, détectée à partir de sa phrase.	« combien je dois payer ce mois ? » → intention <code>consulter_facture</code>
Entité (entity)	Une information précise extraite de la phrase.	« ma facture du contrat 500123 » → entité <code>numero_contrat = 500123</code>
Slot	La « mémoire » du bot : une variable conservée pendant la conversation.	Le numéro de contrat est stocké dans le slot <code>numero_contrat</code> .
Formulaire (form)	Mécanisme qui pose des questions jusqu'à obtenir toutes les informations requises.	<code>facture_form</code> réclame le numéro de contrat s'il n'a pas été donné.
Réponse (response / utterance)	Un message prédéfini que le bot peut envoyer.	<code>utter_payer_facture</code> liste les moyens de paiement.
Custom Action	Du code Python exécuté quand un simple message prédéfini ne suffit pas.	<code>action_consulter_facture</code> interroge la base SQLite.
Règle (rule)	Un enchaînement court et toujours identique : « si intention X, faire Y ».	« si <code>nlu_fallback</code> → appeler <code>action_rag_fallback</code> »
Story	Un exemple de conversation complète, utilisé pour apprendre les dialogues flexibles.	salutation → consultation facture → remerciement → au revoir

3.2. Comment Rasa comprend une phrase : le pipeline NLU

Quand un message arrive, il traverse une chaîne de composants définie dans `config.yml` :

1. **WhitespaceTokenizer** — découpe la phrase en mots.
2. **Featurizers** (Regex, LexicalSyntactic, CountVectors) — transforment les mots en caractéristiques numériques. Le CountVectors en « n-grammes de caractères » (fragments de 1 à 4 lettres) rend le bot **tolérant aux fautes d'orthographe**, fréquentes dans un chat client.
3. **DIETClassifier** — le cœur : un réseau de neurones (Dual Intent and Entity Transformer) entraîné sur nos exemples de `nlu.yml`, qui prédit **l'intention** de la phrase avec un **score de confiance** entre 0 et 1.
4. **RegexEntityExtractor** — extrait le numéro de contrat par expression régulière (6 chiffres commençant par 5). Une regex est fiable à 100 %, contrairement à un apprentissage statistique : choix volontaire

pour une donnée sensible.

5. **FallbackClassifier** — si la meilleure intention a une confiance **inférieure à 0,6**, il remplace la prédiction par l'intention spéciale `nlu_fallback`. C'est **le pont vers le RAG** : une règle associe `nlu_fallback` à l'action qui interroge la documentation.

3.3. Comment Rasa décide quoi répondre : les policies

- **RulePolicy** — applique les règles de `rules.yml` en priorité (dialogue déterministe exigé par le cahier des charges).
- **MemoizationPolicy** — reconnaît les conversations identiques aux stories d'entraînement.
- **TEDPolicy** — réseau de neurones qui généralise à partir des stories pour gérer les conversations qui s'écartent un peu des exemples.

3.4. L'architecture d'exécution de Rasa

Rasa fonctionne avec **deux serveurs** :

- Le **serveur Rasa** (port 5005) : reçoit les messages, exécute NLU + dialogue, expose l'API REST utilisée par notre interface Streamlit.
- Le **serveur d'actions** (port 5055) : exécute notre code Python (`actions.py`). Rasa l'appelle en HTTP quand une custom action est nécessaire (adresse déclarée dans `endpoints.yml`).

4. Comprendre le RAG (Retrieval-Augmented Generation)

Un LLM seul peut « halluciner » : inventer des tarifs, des procédures, des numéros de téléphone. Le RAG corrige ce défaut en forçant le modèle à répondre **uniquement à partir de documents fournis**. Notre pipeline se déroule en deux phases :

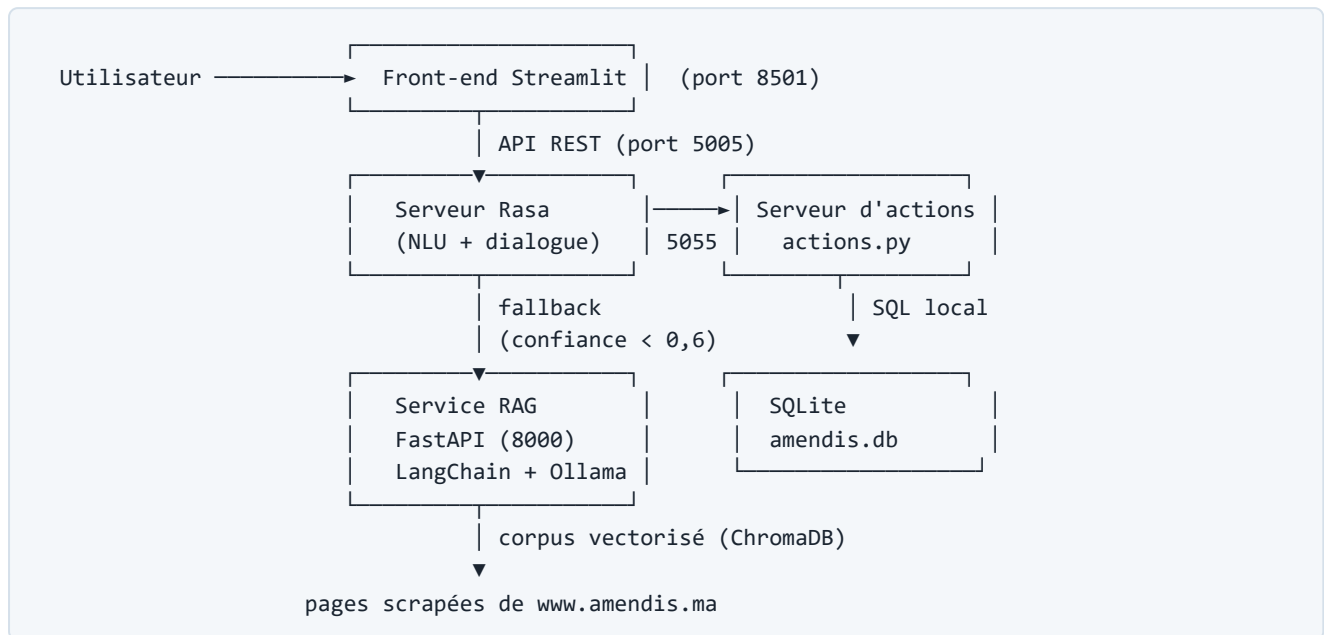
4.1. Phase de préparation (hors ligne, à refaire quand le site change)

1. **Scraping** (`scraper.py`) : un crawler télécharge les pages publiques de www.amendis.ma (FAQ, services, conseils...), en extrait le texte propre et le sauvegarde dans `corpus/` — un fichier texte par page, avec l'URL source en en-tête.
2. **Découpage** (`ingest.py`) : chaque page est découpée en fragments d'environ 800 caractères (avec 120 caractères de chevauchement pour ne pas couper une idée en deux).
3. **Vectorisation** : chaque fragment passe dans le modèle d'embeddings multilingue, qui produit un vecteur de 384 nombres représentant son *sens*. Deux textes qui parlent de la même chose ont des vecteurs proches, même s'ils n'emploient pas les mêmes mots.
4. **Indexation** : les fragments et leurs vecteurs sont stockés dans la base vectorielle **ChromaDB** (dossier `chroma_db/`).

4.2. Phase de réponse (en ligne, à chaque question hors scénario)

1. La question de l'utilisateur est vectorisée à son tour.
2. ChromaDB retrouve les **4 fragments les plus proches** sémantiquement.
3. Ces fragments sont insérés dans un **prompt strict** qui impose au LLM : répondre uniquement à partir des extraits, ne jamais inventer de chiffres, et dire explicitement « je n'ai pas trouvé cette information » si les extraits ne suffisent pas. C'est notre réponse à la contrainte « **zéro hallucination** » du cahier des charges.
4. Le LLM (Ollama, en local) rédige la réponse en français, renvoyée à Rasa puis à l'utilisateur, avec les URL sources.

5. Architecture générale et parcours d'un message



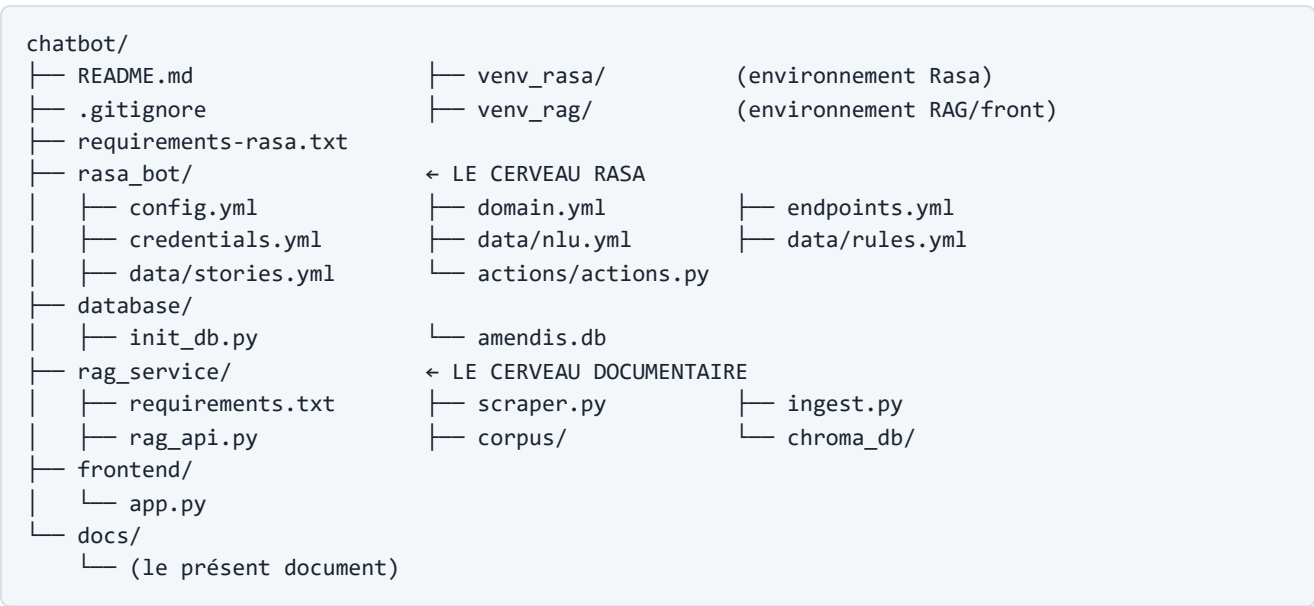
Exemple 1 — scénario métier strict

1. L'utilisateur écrit : « *combien je dois payer ce mois ?* »
2. Le NLU détecte `consulter_facture` (confiance 0,93) — pas de numéro de contrat dans la phrase.
3. Le formulaire `facture_form` s'active et demande le numéro de contrat.
4. L'utilisateur répond « 500123 » ; la regex extrait l'entité, la validation vérifie le format.
5. `action_consulter_facture` interroge SQLite et répond : « *Dernière facture — juin 2026 : 312,80 DH (en attente de paiement)* ».
6. Aucune donnée n'a quitté la machine ; le LLM n'a jamais été sollicité.

Exemple 2 — question documentaire (fallback RAG)

1. L'utilisateur écrit : « *comment résilier mon contrat ?* »
2. Aucune intention ne dépasse 0,6 de confiance → `nlu_fallback`.
3. La règle de fallback déclenche `action_rag_fallback`, qui envoie la question (et uniquement la question) au service RAG.
4. Le RAG retrouve les passages de la FAQ Amendis sur la résiliation et le LLM rédige une réponse fidèle à ces passages, avec l'URL source.

6. L'arborescence du projet, fichier par fichier



Racine du projet

Fichier	Rôle détaillé
README.md	Guide du projet : schéma d'architecture, explication des deux venvs, commandes d'installation pas à pas et commandes de lancement des 4 serveurs.
.gitignore	Liste ce que Git doit ignorer si le projet est versionné : les venvs (des giga-octets reconstructibles), les modèles Rasa entraînés, l'index ChromaDB, la base SQLite — tout ce qui se régénère à partir du code.
requirements-rasa.txt	Dépendances du venv Rasa (rasa 3.6.21 + rasa-sdk 3.6.2), versions épinglées pour la reproductibilité.

rasa_bot/ — le moteur NLU et de dialogue

Fichier	Rôle détaillé
config.yml	La « recette » d'entraînement : le pipeline NLU (tokenizer, featurizers, DIETClassifier 100 époques, RegexEntityExtractor, FallbackClassifier avec seuil 0,6) et les politiques de dialogue (Rule, Memoization, TED). Langue : <code>fr</code> . C'est ici que se règle la sensibilité du fallback : baisser le seuil rend le bot plus « sûr de lui », le monter envoie plus de questions vers le RAG.
domain.yml	La « carte d'identité » du bot : la liste des 8 intentions (5 métier + salutations/remerciements/fallback), l'entité et le slot <code>numero_contrat</code> , le formulaire <code>facture_form</code> , les réponses prédéfinies (textes d'accueil, moyens de paiement, au revoir...) et la déclaration des 4 actions personnalisées. Tout ce qui n'est pas déclaré ici n'existe pas pour Rasa.
data/nlu.yml	Les exemples d'entraînement : ~70 phrases françaises réparties par intention (« combien je dois payer ce mois ? », « il y a une fuite d'eau dans ma rue »...), plus la regex du numéro de contrat. Les entités sont annotées entre crochets dans les exemples. Plus on ajoute d'exemples variés, plus le NLU est précis : c'est le fichier à enrichir en priorité pendant la phase de test.
data/rules.yml	Les dialogues déterministes : saluer → message d'accueil ; payer_facture → explication des moyens de paiement ; signaler_fuite → enregistrement de l'incident ; consulter_facture → activation du formulaire puis appel de l'action SQLite ; nlu_fallback → action_rag_fallback (la règle qui branche le RAG).
data/stories.yml	Trois conversations complètes d'exemple (consultation directe avec numéro, consultation en deux temps, signalement puis question documentaire) pour entraîner la TEDPolicy à gérer les enchaînements flexibles.
endpoints.yml	Indique au serveur Rasa où joindre le serveur d'actions : <code>http://localhost:5055/webhook</code> .
credentials.yml	Active le canal REST , c'est-à-dire l'API HTTP que le front Streamlit utilise pour envoyer les messages des utilisateurs.
actions/actions.py	Le code métier , 4 classes : <code>ValidateFactureForm</code> — valide le format du numéro de contrat saisi (regex <code>5\d{5}</code>) et redemande poliment en cas d'erreur ; <code>ActionConsulterFacture</code> — requête SQL paramétrée (protection contre l'injection SQL) qui joint clients et factures et renvoie la dernière facture avec son statut ; <code>ActionSignalerFuite</code> — insère le signalement dans la table <code>incidents</code> et renvoie un numéro de référence ; <code>ActionRagFallback</code> — envoie la question au service RAG (<code>POST localhost:8000/ask</code>) et relaie la réponse ; gère aussi le cas où le service est éteint.

database/ — la simulation du SI client

Fichier	Rôle détaillé
<code>init_db.py</code>	Script qui (re)crée <code>amendis.db</code> : trois tables — <code>clients</code> (numéro de contrat, nom, adresse, type de service), <code>factures</code> (mois, montant en DH, statut payée/impayée) et <code>incidents</code> (alimentée par les signalements du bot) — puis les remplit avec 5 clients et 7 factures fictifs pour la démonstration.
<code>amendis.db</code>	La base SQLite elle-même : un simple fichier, aucun serveur à installer, embarquée dans Python. Elle simule l'espace client amendisclient.ma.

rag_service/ — le pipeline documentaire

Fichier	Rôle détaillé
<code>requirements.txt</code>	Dépendances du venv RAG (LangChain, ChromaDB, sentence-transformers, FastAPI, Streamlit, BeautifulSoup...).
<code>scraper.py</code>	Le crawler de <code>www.amendis.ma</code> . Il part de pages graines (accueil, FAQ), suit uniquement les liens internes en <code>/fr</code> , se limite à 60 pages, attend 1 seconde entre chaque requête (politesse), supprime les balises non informatives (menus, scripts, pieds de page) et sauvegarde chaque page en fichier texte dans <code>corpus/</code> avec son URL source en première ligne — ce qui permettra de citer les sources dans les réponses.
<code>ingest.py</code>	La vectorisation : charge le corpus, le découpe en fragments de 800 caractères (chevauchement 120), calcule les embeddings avec le modèle multilingue <code>paraphrase-multilingual-MiniLM-L12-v2</code> (exécuté en local) et construit l'index ChromaDB persistant. À relancer après chaque mise à jour du corpus — c'est la fonction « Administrateur » du cahier des charges (section 3.2.1).
<code>rag_api.py</code>	Le service de réponse (FastAPI, port 8000). Endpoint <code>POST /ask</code> : vectorise la question, récupère les 4 fragments les plus pertinents dans ChromaDB, les insère dans le prompt strict anti-hallucination, appelle le LLM local via Ollama (température 0 = réponses les plus factuelles possibles) et renvoie la réponse avec les URL sources. Endpoint <code>GET /health</code> pour vérifier que le service tourne.
<code>corpus/</code>	Les pages scrapées (un fichier .txt par page). C'est la « base documentaire » du bot.
<code>chroma_db/</code>	L'index vectoriel persistant généré par <code>ingest.py</code> (créé au premier lancement).

frontend/ — l'interface utilisateur

Fichier	Rôle détaillé
<code>app.py</code>	L'interface de chat Streamlit : zone de saisie libre, historique de la conversation avec bulles utilisateur/assistant, barre latérale de sujets fréquents cliquables (exigence « accueil automatisé » du cahier des charges, section 3.1.1), et identifiant de session unique (UUID) pour que Rasa suive chaque conversation séparément. Communique avec Rasa via <code>POST localhost:5005/webhooks/rest/webhook</code> et affiche un message clair si le serveur n'est pas lancé.

7. Réponse aux contraintes du cahier des charges

Contrainte	Comment le projet y répond
Sécurité des données	Les numéros de contrat et montants sont traités uniquement dans <code>actions.py</code> + SQLite, en local. Le LLM ne reçoit que le texte des questions générales. Requêtes SQL paramétrées (anti-injection), numéro de contrat validé par regex avant toute requête.
Zéro hallucination	Prompt strict (répondre uniquement d'après les extraits, avec d'ignorance obligatoire sinon), température 0, LLM local, et sources citées avec chaque réponse.
Délai de 4 semaines	Squelette complet opérationnel dès la semaine 1 ; les semaines suivantes servent à enrichir les données NLU, affiner le corpus et tester.
Déploiement optionnel	L'API REST de Rasa est le point d'intégration standard : le front Streamlit pourra être remplacé par un widget sur le site officiel sans toucher au cœur du bot.

8. État actuel et prochaines étapes

Fait : Python 3.10.11 installé · structure complète du projet écrite (14 fichiers) · base SQLite créée et peuplée · installation des deux venvs lancée.

1. Finaliser l'installation des deux venvs (en cours).
2. Installer **Ollama** et télécharger le modèle `llama3.2`.
3. Lancer le scraper puis l'ingestion : `python rag_service/scrapper.py` puis `python rag_service/ingest.py`.
4. Entraîner le modèle Rasa : `rasa train` (dans `rasa_bot/`).
5. Tester en conditions réelles : les 4 serveurs + conversation complète, puis enrichir `nlu.yml` avec les formulations mal comprises.